

## Шаблоны классов

В прошлом семестре вы уже знакомились с шаблонами функций и структур. Теперь, познакомившись с классами, мы можем создавать шаблоны и для классов.

Выглядит шаблон класс так же, как и шаблон структуры. Если методы используют шаблонные параметры и определяются внутри класса, то перед их определением не нужно писать `template <...>`, т.к. это уже написано перед объявлением класса. Если метод определяется вне класса, то перед определением, аналогично шаблону функции, необходимо написать `template <...>` (примеры 1 и 2).

Далее нужно вспомнить несколько моментов:

1. Для удобства использования классов и ускорения компиляции, мы разносили классы на два файла: заголовочный файл (.h) и файл с исходным кодом (.cpp).
2. Заголовочные файлы подключаются в файлы исходного кода (или друг в друга, а затем в файлы исходного кода) директивой `#include`, при компиляции эта строка заменяется содержимым подключаемого файла, а непосредственно компилируются файлы с исходным кодом, каждый из которых превращается в свой объектный файл.
3. Компилятор в случае наличия вызова шаблона класса/функции по шаблону генерирует класс/функцию.

Собирая эти моменты воедино, получаем следующие выводы:

- 1) код файла, где объявлен класс и/или функции, будет скопирован (т.е. подключен) в какой-нибудь файл с исходным кодом (например, «Source.cpp»), который скомпилирован в объектный файл с учётом параметров шаблонов;
- 2) определения методов и внешних функций, вынесенные в отдельный файл с исходным кодом, компилируются отдельно и независимо в свой объектный файл. Поскольку в этот файл не поступала информация параметрах шаблона, компилятор не генерирует объявления;
- 3) при линковке (связывании) линкер не находит реализацию для объявления конкретного типа данных и выдаёт ошибку.

Схема сборки проекта приведена на рис. 1.

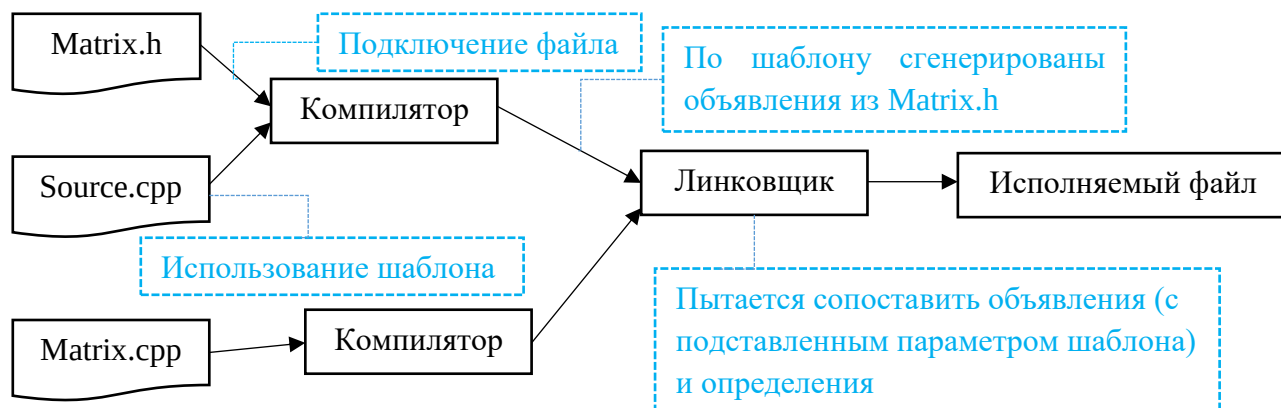


Рис. 1. Схема сборки проекта

### Пример 1. Содержимое файла «Matrix.h»

```
1  #pragma once
2
3  template <class T>
4  class Matrix
5  {
6      size_t n, m;
7      T** values;
8  public:
9      Matrix(const int& n1 = 0, const int& m1 = 0);
10     void print();
11     ~Matrix();
12 };
```

### Пример 2. Содержимое файла «Matrix.cpp»

```
1  #include "Matrix.h"
2  #include <iostream>
3
4  template <class T>
5  Matrix<T>::Matrix(const int& n1, const int& m1) : n(n1), m(m1), values(nullptr)
6  {
7      if ((n < 0) || (m < 0)) // Коррекция значений
8          n = m = -1;
9      else
10     {
11         values = new T* [n] ;
12         for (size_t i = 0; i < n; ++i)
13         {
14             values[i] = new T [m];
15             for (size_t j = 0; j < m; ++j)
16                 values[i][j] = 0;
17         }
18     }
19 }
20
21 template <class T>
22 Matrix<T>::print()
23 {
24     if (n == 0) // Коррекция значений
25         std::cout << "Matrix is empty\n";
26     else
27     {
28         for (size_t i = 0; i < n; ++i)
29         {
30             for (size_t j = 0; j < m; ++j)
31                 std::cout << values[i][j] << " ";
32             std::cout << std::endl;
33         }
34     }
35 }
36
37 template <class T>
38 Matrix<T>::~~Matrix()
39 {
40     if (values != nullptr)
41     {
42         for (size_t i = 0; i < n; ++i)
43             delete[] values[i];
44         delete[] values;
45     }
46 }
47
48
```

Таким образом, если проект содержит всего 3 файла с кодом на C++: «Matrix.h», «Matrix.cpp» и «Source.cpp» (пример 3), при линковке нас ожидает ошибка (рис. 2).

Пример 3. Содержимое файла «Source.cpp»

```
1  #include "Matrix.h"
2
3  int main()
4  {
5      Matrix<int> m;
6      m.print();
7      return 0;
8  }
```

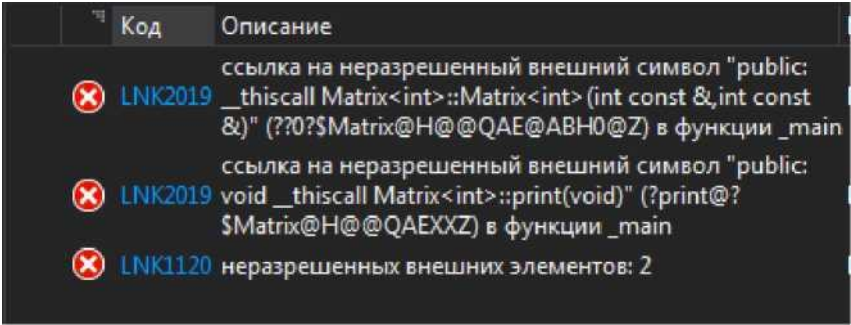


Рис. 2. Ругается линкер Visual Studio

Но надо же как-то работать с шаблонами, правда?

Первое и самое очевидное решение - «откатиться» назад разработке проектов и засунуть весь код класса в один файл. Поскольку подключаются именно заголовочные файлы, то в «Matrix.h». Подводные камни вы знаете: если файл большой и много где подключается, возникает много избыточного кода, который замедляет компиляцию, а если проект содержит несколько cpp-файлов, в которых вы используете заголовочный файл, то все их нужно перекомпилировать при внесении малейших изменений в реализацию класса.

Второе решение – ещё больший «откат»: засунуть весь код класса в отдельный файл исходного кода «Matrix.cpp» и подключать его. Проблема с подключением и перекомпиляцией та же, что и в предыдущем решении.

Третье решение – явная генерация классов/функций для нужного типа данных в отдельном файле исходного кода. Его код приведён в примере 4.

Пример 4. Содержимое файла «IntMatrix.cpp»

```
1  #include "Matrix.h"
2  #include "Matrix.cpp"
3
4  template class Matrix<int>;
```

Добавление в проект «IntMatrix.cpp» не требует изменения кода остальных файлов проекта. Строка 4 указывает компилятору на создание в этом месте класса с типом `int` на месте `T`. Поскольку требуется также определить методы с параметром `int`, требуется также подключить и файл с исходным кодом.

### **Третье решение является предпочтительным.**

Стоит отметить, что файл «IntMatrix.cpp» не является частью класса-матрицы. Он содержит код, связанный с использованием класса, и его создание возложено на конечного программиста (программиста, использующего класс `Matrix`), который вовсе не обязательно является его автором.

Если класс из примера задумывался как матрица в математическом смысле, можно ожидать его использование для таких числовых типов, как `int` и `double`. Таким образом, файл «Matrix.cpp» может быть дополнен строками «`template class Matrix<int>;`» и «`template class Matrix<double>;`». Тогда при компиляции файла «Matrix.cpp» будут созданы реализации для типов `int` и `double`. Это породит избыточный код, если конечный программист не будет использовать обе реализации, однако упрощает эксплуатацию класса для конечного программиста.